PROGRAMMING FOR AI

LAB MANUAL

SUBMITTED BY: YUMNA IRFAN

SUBMITTED TO: SIR SAMRAIZ

REGESTRATION NO: 2023-BS-AI-021

DEPARTMENT: ARTIFICIAL INTELLIGENCE

Table of Contents

LAB 1

Q1. Banking Transaction Validator with PIN

Simulate a bank ATM.

User enters a 4-digit PIN (check if valid).

Display account balance and ask for withdrawal amount.

If valid → deduct + update balance.

If invalid attempts ≥ 3 → block card.

Use operators to compute service charges (2% per withdrawal)**.**

[2]:
```
PIN = "1234"
balance = 500.00
attempts = 0
MAX_ATTEMPTS = 3
CHARGE_RATE = 0.02 # 2%

print("Welcome to Quick ATM!")

while attempts < MAX_ATTEMPTS:
    user_pin = input("Enter PIN: ")

    if user_pin == PIN:
        print("\nPIN accepted.")
        break

    attempts += 1
    if attempts == MAX_ATTEMPTS:
        print("🚨 Card blocked.")
        exit()
    else:
        print(f"Incorrect. {MAX_ATTEMPTS - attempts} attempts left.")

try:
    print(f"Balance: ${balance:,.2f}")
    amount = float(input("Enter withdrawal amount: "))
except ValueError:
    print("Invalid amount input.")
    exit()

if amount <= 0:
    print("Amount must be positive.")
elif amount > balance:
    print("Insufficient funds.")
else:

    charge = amount * CHARGE_RATE
    total_debit = amount + charge

    if total_debit <= balance:
        balance -= total_debit
        print(f"\nCharge (2%): ${charge:,.2f}")
        print(f"Success! Withdrew ${amount:,.2f}")
        print(f"New Balance: ${balance:,.2f}")
    else:
        print("Insufficient funds (need to cover withdrawal + charge).")

print("Thank you!")
```

```
Welcome to Quick ATM!
Enter PIN:  1234

PIN accepted.
Balance: $500.00
Enter withdrawal amount:  200

Charge (2%): $4.00
Success! Withdrew $200.00
New Balance: $296.00
Thank you!
```

Q2. Grocery Store Inventory System

Maintain item names, stock, and prices in a dictionary of dictionaries.

User selects items with quantity.

If stock available, deduct from inventory.

Apply discount slabs (10%, 15%, 20%).

Print final bill with tax (5%).

[2]:
```
inventory = {
    'apple': {'stock': 50, 'price': 1.50},
    'banana': {'stock': 30, 'price': 0.50},
    'milk': {'stock': 10, 'price': 3.00}
}

TAX_RATE = 0.05

def get_discount_rate(total):
    if total >= 50:
        return 0.20
    elif total >= 20:
        return 0.15
    else:
        return 0.10

print("Welcome to the Grocery Store!")
print("Available Items:", list(inventory.keys()))

item_name = input("Enter item name: ").lower()
if item_name not in inventory:
    print("Item not found.")
    exit()

try:
    quantity = int(input("Enter quantity: "))
except ValueError:
    print("Invalid quantity.")
    exit()

if quantity > inventory[item_name]['stock']:
    print(f"Insufficient stock. Only {inventory[item_name]['stock']} available.")
    exit()

inventory[item_name]['stock'] -= quantity

price = inventory[item_name]['price']
subtotal = quantity * price
discount_rate = get_discount_rate(subtotal)
discount_amount = subtotal * discount_rate
tax_amount = (subtotal - discount_amount) * TAX_RATE
final_total = subtotal - discount_amount + tax_amount

print(" Final Bill ")
print(f"Item: {item_name.title()} (x{quantity})")
print(f"Price per unit: ${price:,.2f}")
print(f"Subtotal: ${subtotal:,.2f}")
print(f"Discount ({discount_rate*100:.0f}%): -${discount_amount:,.2f}")
print("-" * 20)
print(f"Tax (5%): +${tax_amount:,.2f}")
print(f"TOTAL PAYABLE: ${final_total:,.2f}")
print("-" * 20)
print(f"Updated {item_name.title()} Stock: {inventory[item_name]['stock']}")
```

```
Welcome to the Grocery Store!
Available Items: ['apple', 'banana', 'milk']
Enter item name:  banana
Enter quantity:  3
 Final Bill
Item: Banana (x3)
Price per unit: $0.50
Subtotal: $1.50
Discount (10%): -$0.15
--------------------
Tax (5%): +$0.07
TOTAL PAYABLE: $1.42
--------------------
Updated Banana Stock: 27
```

Q3. Student Grading with Subject Weightage

Input marks for 5 subjects where each subject has different weightage (stored in a tuple).

Compute weighted average.

Assign grade (A/B/C/D/Fail).

Also, display whether the student is eligible for scholarship (≥80% weighted).

[4]:
```
SUBJECTS = ["Math", "Physics", "Chem", "English", "CS"]
WEIGHTS = (0.30, 0.25, 0.20, 0.15, 0.10)
NUM_SUBJECTS = len(SUBJECTS)
marks = []
total_weighted_score = 0.0

print(" Student Grading System ")
print("Enter marks (out of 100) for the 5 subjects:")

for i in range(NUM_SUBJECTS):
    try:
        mark = float(input(f"Enter {SUBJECTS[i]} Mark: "))
        if not (0 <= mark <= 100):
            print("Mark must be between 0 and 100. Exiting.")
            exit()
        marks.append(mark)

        weighted_mark = mark * WEIGHTS[i]
        total_weighted_score += weighted_mark

    except ValueError:
        print("Invalid input. Please enter a number. Exiting.")
        exit()

weighted_average = total_weighted_score
print("-" * 30)
print(f"Total Weighted Score: {weighted_average:.2f}%")

grade = ""
if weighted_average >= 90:
    grade = "A"
elif weighted_average >= 80:
    grade = "B"
elif weighted_average >= 70:
    grade = "C"
elif weighted_average >= 60:
    grade = "D"
else:
    grade = "Fail"

print(f"Final Grade Assigned: {grade}")

if weighted_average >= 80.0:
    print(" Scholarship Eligibility: Student IS eligible for scholarship (>=80%).")
else:
    print(" Scholarship Eligibility: Student is NOT eligible for scholarship.")

print("-" * 30)
```

```
 Student Grading System
Enter marks (out of 100) for the 5 subjects:
Enter Math Mark:  67
Enter Physics Mark:  87
Enter Chem Mark:  55
Enter English Mark:  96
Enter CS Mark:  80
------------------------------
Total Weighted Score: 75.25%
Final Grade Assigned: C
 Scholarship Eligibility: Student is NOT eligible for scholarship.
```

Q4. BMI + Health Recommendation

Extend BMI program:

Take weight, height, and age.

Compute BMI.

Based on BMI and age, give personalized health suggestions (e.g., diet plan for obese, exercise plan for underweight)

[5]:
```
def calculate_bmi_recommendation():
    print("--- BMI & Health Advisor ---")

    try:
        weight = float(input("Enter weight (kg): "))
        height = float(input("Enter height (m): "))
        age = int(input("Enter age: "))
    except ValueError:
        print("Invalid input. Please enter numbers.")
        return

    if height <= 0:
        print("Height must be positive.")
        return

    bmi = weight / (height ** 2)

    recommendation = ""

    if bmi < 18.5:
        category = "Underweight"
        recommendation = "Focus on increasing caloric intake with nutrient-dense foods."
    elif 18.5 <= bmi < 24.9:
        category = "Normal weight"
        recommendation = "Maintain current healthy eating habits and regular activity."
    elif 25 <= bmi < 29.9:
        category = "Overweight"
        recommendation = "Incorporate more cardiovascular exercise and reduce processed foods."
    else:
        category = "Obese"
        recommendation = "Consult a doctor for a structured diet plan and begin an exercise plan for obese."

    if category in ["Overweight", "Obese"] and age >= 50:
        recommendation += " Since you are over 50, ensure exercises are low-impact (e.g., swimming, walking)."
    elif category == "Underweight" and age < 18:
        recommendation += " Consult a pediatrician for specific dietary advice."

    print("-" * 35)
    print(f"Your BMI is: {bmi:.2f}")
    print(f"Category: {category}")
    print("-" * 35)
    print(f"Personalized Suggestion: {recommendation}")

calculate_bmi_recommendation()
```

```
--- BMI & Health Advisor ---
Enter weight (kg):  65
Enter height (m):  187
Enter age:  24
-----------------------------------
Your BMI is: 0.00
Category: Underweight
-----------------------------------
Personalized Suggestion: Focus on increasing caloric intake with nutrient-dense foods.
```

Q5. Telecom Prepaid Recharge System

Maintain call rate/min, SMS rate, and data/MB in a dictionary.

User selects usage (calls, SMS, data).

Deduct accordingly from balance.

If balance < 20% of initial → print recharge reminder.

Apply bonus balance if recharge > 1000.

[6]:
```
RATES = {
    'call': 1.50,
    'sms': 0.10,
    'data': 0.05
}
INITIAL_BALANCE = 500.00
balance = INITIAL_BALANCE

def check_recharge_reminder():
    if balance < (INITIAL_BALANCE * 0.20):
        print(" LOW BALANCE ALERT: Please recharge! Your balance is below 20% of the initial amount.")

def apply_bonus(recharge_amount):
    if recharge_amount > 1000:
        bonus = recharge_amount * 0.10
        return bonus
    return 0

print(" Telecom Usage Tracker ")
print(f"Starting Balance: Rs{balance:,.2f}")

try:
    calls = float(input("Enter Call Usage (minutes): "))
    sms = int(input("Enter SMS Usage (count): "))
    data = float(input("Enter Data Usage (MB): "))
except ValueError:
    print("Invalid input. Please enter numbers.")
    exit()

call_cost = calls * RATES['call']
sms_cost = sms * RATES['sms']
data_cost = data * RATES['data']
total_cost = call_cost + sms_cost + data_cost

print("-" * 30)
print(f"Call Cost: Rs{call_cost:,.2f}")
print(f"SMS Cost: Rs{sms_cost:,.2f}")
print(f"Data Cost: Rs{data_cost:,.2f}")
print(f"Total Usage Cost: Rs{total_cost:,.2f}")
print("-" * 30)

if total_cost <= balance:
    balance -= total_cost
    print(f"Balance after usage: Rs{balance:,.2f}")
    check_recharge_reminder()
else:
    print("Usage exceeds current balance. Please recharge.")
    exit()

if balance < 200:
    try:
        recharge_amount = float(input("\nEnter Recharge Amount: Rs"))
    except ValueError:
        print("Invalid recharge amount.")
        exit()

    bonus = apply_bonus(recharge_amount)
    balance += recharge_amount + bonus

    if bonus > 0:
        print(f" Bonus applied! You received Rs{bonus:,.2f} bonus.")

    print(f"New Final Balance: Rs{balance:,.2f}")

print("\nSession Ended.")
```

```
 Telecom Usage Tracker
Starting Balance: Rs500.00
Enter Call Usage (minutes):  30
Enter SMS Usage (count):  13
Enter Data Usage (MB):  25
------------------------------
Call Cost: Rs45.00
SMS Cost: Rs1.30
Data Cost: Rs1.25
Total Usage Cost: Rs47.55
------------------------------
Balance after usage: Rs452.45

Session Ended.
```

Lab 2

Q6. E-Commerce Discount Engine with Membership Levels

User inputs bill amount + membership type (Regular, Gold, Platinum).

Apply different discount rules (Gold +10%, Platinum +15%).

If bill > 10000, apply additional 5% flat.

Show itemized receipt with tax breakdown.

[1]:
```python
def generate_receipt(bill_amount, membership_type):
    TAX_RATE = 0.18
    discount = 0

    if membership_type.lower() == "gold":
        discount += 0.10
    elif membership_type.lower() == "platinum":
        discount += 0.15

    if bill_amount > 10000:
        discount += 0.05

    discount_amount = bill_amount * discount
    discounted_price = bill_amount - discount_amount
    tax_amount = discounted_price * TAX_RATE
    final_amount = discounted_price + tax_amount

    print(" E-Commerce Receipt ")
    print(f"Bill Amount         : ₹{bill_amount:.2f}")
    print(f"Membership Type     : {membership_type}")
    print(f"Discount Applied    : {discount * 100:.0f}%")
    print(f"Discount Amount     : -₹{discount_amount:.2f}")
    print(f"Price After Discount: ₹{discounted_price:.2f}")
    print(f"Tax (18%)           : ₹{tax_amount:.2f}")
    print(f"Final Amount Payable: ₹{final_amount:.2f}")

generate_receipt(12000, "Gold")
```

```
 E-Commerce Receipt
Bill Amount         : ₹12000.00
Membership Type     : Gold
Discount Applied    : 15%
Discount Amount     : -₹1800.00
Price After Discount: ₹10200.00
Tax (18%)           : ₹1836.00
Final Amount Payable: ₹12036.00
```

Q7. Hospital Emergency Unit with Multi-Condition Triage

Take patient details: temperature, pulse, oxygen, blood pressure.

Classify into categories: Stable, Observation, Urgent, Critical.

If critical → suggest "Admit in ICU."

If urgent → suggest "Immediate doctor attention."

Use nested conditions.

```
temp = float(input("Enter Temperature (°C): "))
pulse = int(input("Enter Pulse (bpm): "))
oxy = int(input("Enter Oxygen Level (%): "))
bp = int(input("Enter Systolic BP: "))

print("\n--- Patient Triage Result ---")

if oxy < 85 or bp < 80:
    print("Category: Critical")
    print("🚨 Suggestion: Admit in ICU immediately!")
elif oxy < 92 or pulse > 120 or temp > 39:
    print("Category: Urgent")
    print("⚠️ Suggestion: Immediate doctor attention needed!")
elif (37.5 <= temp <= 38.5) or (100 <= pulse <= 120) or (bp < 100):
    print("Category: Observation")
    print("👀 Suggestion: Keep under observation.")
else:
    print("Category: Stable")
    print("✅ Suggestion: Normal monitoring.")
```

```
Enter Temperature (°C): 40
Enter Pulse (bpm): 140
Enter Oxygen Level (%): 80
Enter Systolic BP: 70

--- Patient Triage Result ---
Category: Critical
🚨 Suggestion: Admit in ICU immediately!
```

Q8. Online Examination Grader with Negative Marking

Take total marks, obtained marks, and number of wrong answers.

Deduct 0.25 per wrong answer.

Compute percentage.

Assign division + eligibility for scholarship.

If Fail → suggest "Repeat Exam."

```
t = float(input("Total Marks: "))
o = float(input("Obtained Marks: "))
w = int(input("Wrong Answers: "))

f = max(0, o - w*0.25)          # final marks after penalty
p = (f/t)*100                   # percentage

if p >= 80: d, s = "Distinction", "Eligible"
elif p >= 60: d, s = "First Division", "Not Eligible"
elif p >= 45: d, s = "Second Division", "Not Eligible"
elif p >= 33: d, s = "Third Division", "Not Eligible"
else: d, s = "Fail", "Repeat Exam"

print(f"\nMarks: {f}/{t}, %: {p:.2f}, Division: {d}")
if d == "Fail":
    print("Suggestion:", s)
else:
    print("Scholarship:", s)
```

```
Total Marks: 500
Obtained Marks: 420
Wrong Answers: 8

Marks: 418.0/500.0, %: 83.60, Division: Distinction
Scholarship: Eligible
```

Q9. Movie Ticket Booking with Age & Timing Policy

User inputs: age, showtime, ticket type (Normal/3D/IMAX).

Apply child restrictions (No late-night for <12).

Apply senior citizen discount (30%).

Apply peak hour charges (10%).

Print final ticket price.

```
prices = {"normal": 300, "3d": 400, "imax": 600}

age = int(input("Enter Age: "))
time = int(input("Enter Show Time (24hr format): "))
ttype = input("Ticket Type (Normal/3D/IMAX): ").lower()

price = prices[ttype]

# Restrictions
if age < 12 and (time >= 22 or time < 8):
    print("Not Allowed: Children cannot book late-night shows.")
else:
    # Senior Citizen Discount
    if age >= 60:
        price *= 0.7   # 30% off

    # Peak Hour (6pm-10pm)
    if 18 <= time <= 22:
        price *= 1.1   # 10% extra

    print(f"Final Ticket Price: Rs.{price:.2f}")
```

```
Enter Age: 10
Enter Show Time (24hr format): 23
Ticket Type (Normal/3D/IMAX): Normal
Not Allowed: Children cannot book late-night shows.
```

Q10. Weather-based Outfit + Travel Suggestion

Input temperature, weather (Rainy/Sunny/Cold), event type (Casual/Business/Party).

Suggest outfit + accessories.

If Rainy → umbrella; If Cold → gloves.

If event is Business + temperature > 35 → recommend "light formal suit."

[2]:
```
temperature = float(input("Enter temperature: "))
weather = input("Enter weather (Rainy/Sunny/Cold): ").lower()
event = input("Enter event type (Casual/Business/Party): ").lower()

outfit = ""
accessories = []
travel = ""

if event == "business":
    if temperature > 35:
        outfit = "Light formal suit"
    else:
        outfit = "Formal wear"
elif event == "party":
    outfit = "Party outfit"
else:
    outfit = "Casual wear"

if weather == "rainy":
    accessories.append("Umbrella")
    travel = "Use covered transport"
elif weather == "cold":
    accessories.append("Gloves")
    travel = "Use warm transport"
else:
    travel = "Any transport is fine"

print("\n Recommendation ")
print("Outfit:", outfit)
print("Accessories:", ", ".join(accessories) if accessories else "None")
print("Travel Suggestion:", travel)
```

```
Enter temperature:  24
Enter weather (Rainy/Sunny/Cold):  sunny
Enter event type (Casual/Business/Party):  business

 Recommendation
Outfit: Formal wear
Accessories: None
Travel Suggestion: Any transport is fine
```

Lab 3

Q11. ATM Withdrawal with Note Counting + Receipt

Take withdrawal amount.

Compute minimum number of notes (1000, 500, 100, 50, 10, 1).

Also compute service charge = 0.5%.

Print receipt with remaining balance after deduction.

[3]:
```
balance = float(input("Enter account balance: "))
amount = int(input("Enter withdrawal amount: "))

service_charge = amount * 0.005
total_deduction = amount + service_charge

if total_deduction > balance:
    print("Insufficient balance")
else:
    notes = [1000, 500, 100, 50, 10, 1]
    note_count = {}

    remaining = amount
    for note in notes:
        note_count[note] = remaining // note
        remaining %= note

    new_balance = balance - total_deduction

    print("\n ATM Receipt ")
    print(f"Withdrawal Amount : ₹{amount}")
    print("Notes Dispensed:")
    for note in notes:
        if note_count[note] > 0:
            print(f"₹{note} x {note_count[note]}")
    print(f"Service Charge    : ₹{service_charge:.2f}")
    print(f"Remaining Balance : ₹{new_balance:.2f}")
```

```
Enter account balance:  50000
Enter withdrawal amount:  20000

 ATM Receipt
Withdrawal Amount : ₹20000
Notes Dispensed:
₹1000 x 20
Service Charge    : ₹100.00
Remaining Balance : ₹29900.00
```

Q12. Library Fine Calculator with Membership Levels

Input days late.

Apply fines (10, 20, 50/day).

If days > 30 → "Membership Cancelled."

If user is Premium member, reduce fine by 50%.

Print detailed fine slip.

[4]:
```
days_late = int(input("Enter number of days late: "))
membership = input("Enter membership type (Normal/Premium): ").lower()

if days_late <= 10:
    fine_per_day = 10
elif days_late <= 20:
    fine_per_day = 20
else:
    fine_per_day = 50

total_fine = days_late * fine_per_day

if membership == "premium":
    total_fine *= 0.5

print("\n Library Fine Slip ")
print(f"Days Late        : {days_late}")
print(f"Fine Per Day     : ₹{fine_per_day}")

if days_late > 30:
    print("Status           : Membership Cancelled")
else:
    print("Status           : Active")

print(f"Membership Type  : {membership.capitalize()}")
print(f"Total Fine       : ₹{total_fine:.2f}")
```

```
Enter number of days late:  4
Enter membership type (Normal/Premium):  normal

 Library Fine Slip
Days Late        : 4
Fine Per Day     : ₹10
Status           : Active
Membership Type  : Normal
Total Fine       : ₹40.00
```

Q13. Sales Analysis Dashboard

Store monthly sales in a list (12 entries).

Print max, min, average.

Print months above average.

Use loop + condition to print “Growth” or “Decline” compared to last month.

```
sales = [12000, 15000, 14000, 16000, 18000, 20000, 22000, 21000, 23000, 25000, 24000, 26000]
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
          "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]

max_sale = sales[0]
min_sale = sales[0]
total = 0

for s in sales:
    if s > max_sale:
        max_sale = s
    if s < min_sale:
        min_sale = s
    total += s

average = total / len(sales)

print("Max Sale:", max_sale)
print("Min Sale:", min_sale)
print("Average Sale:", average)

print("\nMonths Above Average:")
for i in range(len(sales)):
    if sales[i] > average:
        print(months[i])

print("\nMonthly Trend:")
for i in range(1, len(sales)):
    if sales[i] > sales[i - 1]:
        print(months[i], ": Growth")
    else:
        print(months[i], ": Decline")
```

```
Max Sale: 26000
Min Sale: 12000
Average Sale: 19666.666666666668

Months Above Average:
Jun
Jul
Aug
Sep
Oct
Nov
Dec

Monthly Trend:
Feb : Growth
Mar : Decline
Apr : Growth
May : Growth
Jun : Growth
Jul : Growth
Aug : Decline
Sep : Growth
Oct : Growth
Nov : Decline
Dec : Growth
```

Q14. Multiplication Puzzle with Random Blanks
Generate a multiplication table (1–10).
Randomly replace 3 results with "??".
Ask user to guess values.
Track score. Give performance remark (Excellent/Good/Try Again).

[6]:
```
import random

table = []
blanks = set()
score = 0

while len(blanks) < 3:
    blanks.add((random.randint(1, 10), random.randint(1, 10)))

print(" Multiplication Puzzle ")

for i in range(1, 11):
    row = []
    for j in range(1, 11):
        if (i, j) in blanks:
            row.append("??")
        else:
            row.append(str(i * j))
    table.append(row)
    print("\t".join(row))

print("\nGuess the missing values:")

for (i, j) in blanks:
    guess = int(input(f"{i} x {j} = "))
    if guess == i * j:
        score += 1

print("\nScore:", score, "/ 3")

if score == 3:
    print("Performance: Excellent")
elif score == 2:
    print("Performance: Good")
else:
    print("Performance: Try Again")
```

```
 Multiplication Puzzle
1       2       3       4       5       6       7       8       9       10
2       4       6       8       10      12      14      16      ??      20
3       6       9       12      15      18      21      24      27      30
4       8       12      16      20      24      28      32      36      40
5       10      15      20      25      30      35      40      45      50
6       12      18      24      30      36      42      48      54      60
7       14      21      28      35      42      49      56      63      70
8       16      24      32      40      48      56      64      72      80
9       18      27      36      45      54      ??      72      81      90
10      20      ??      40      50      60      70      80      90      100

Guess the missing values:
2 x 9 =  18
9 x 7 =  56
10 x 3 =  30

Score: 2 / 3
Performance: Good
```

Q15. Password Strength + Re-entry Attempts

Check password rules (length, digit, upper, lower, special).

If weak → ask user to re-enter.

Allow max 3 attempts.

If still weak → "Account Locked."

[7]:
```
import re

attempts = 3
strong = False

while attempts > 0:
    password = input("Enter password: ")

    length_ok = len(password) >= 8
    digit_ok = re.search(r"\d", password)
    upper_ok = re.search(r"[A-Z]", password)
    lower_ok = re.search(r"[a-z]", password)
    special_ok = re.search(r"[!@#$%^&*()_+=\-]", password)

    if length_ok and digit_ok and upper_ok and lower_ok and special_ok:
        strong = True
        break
    else:
        attempts -= 1
        print("Weak password. Attempts left:", attempts)

if strong:
    print("Password accepted")
else:
    print("Account Locked")
```

```
Enter password:  12345
Weak password. Attempts left: 2
Enter password:  563298hh
Weak password. Attempts left: 1
Enter password:  yumna@1122!
Weak password. Attempts left: 0
Account Locked
```

Lab 4

Q16. Electricity Bill with Slabs & Penalty

Function calculate_bill(units, late_payment=False):

Apply slab rates (5/7/10 Rs).

If late_payment → add 15% penalty.

Return bill breakdown.

```
def calculate_bill(units, late_payment=False):
    bill = 0

    if units <= 100:
        bill = units * 5
    elif units <= 200:
        bill = (100 * 5) + (units - 100) * 7
    else:
        bill = (100 * 5) + (100 * 7) + (units - 200) * 10

    penalty = bill * 0.15 if late_payment else 0
    total = bill + penalty

    return {
        "Units Consumed": units,
        "Base Amount": bill,
        "Penalty": penalty,
        "Total Bill": total
    }


result = calculate_bill(250, late_payment=True)
for k, v in result.items():
    print(f"{k}: ₹{v:.2f}" if isinstance(v, (int, float)) else f"{k}: {v}")
```

```
Units Consumed: ₹250.00
Base Amount: ₹1700.00
Penalty: ₹255.00
Total Bill: ₹1955.00
```

Q17. Cricket Match Predictor with Multiple Conditions

Function predict_score(runs, balls, wickets, overs_left):

Compute strike rate & run rate.

If run rate < 5 and wickets > 6 → "Losing."

If run rate > 7 and wickets < 5 → "Winning."

Else → "Balanced."

[9]:
```
def predict_score(runs, balls, wickets, overs_left):
    if balls == 0 or overs_left == 0:
        return "Invalid input"
    strike_rate = (runs / balls) * 100
    run_rate = runs / (20 - overs_left)
    if run_rate < 5 and wickets > 6:
        return "Losing"
    elif run_rate > 7 and wickets < 5:
        return "Winning"
    else:
        return "Balanced"

runs = int(input("Enter current runs: "))
balls = int(input("Enter balls faced: "))
wickets = int(input("Enter wickets lost: "))
overs_left = float(input("Enter overs left: "))

status = predict_score(runs, balls, wickets, overs_left)
print("Match Status:", status)
```

```
Enter current runs:  5
Enter balls faced:  3
Enter wickets lost:  0
Enter overs left:  10
Match Status: Balanced
```

Q18. Hotel Booking with Tax & Discount

Function book_room(type, days, is_member):

Standard: 2000/day, Deluxe: 4000/day, Suite: 6000/day.

If days > 7 → 20% discount.

If member → extra 10% discount.

Add 12% tax.

Return bill slip.

[10]:
```
def book_room(room_type, days, is_member):
    rates = {"Standard": 2000, "Deluxe": 4000, "Suite": 6000}
    base = rates.get(room_type, 0) * days
    discount = 0
    if days > 7:
        discount += 0.2 * base
    if is_member:
        discount += 0.1 * base
    subtotal = base - discount
    tax = 0.12 * subtotal
    total = subtotal + tax
    return {
        "Room Type": room_type,
        "Days": days,
        "Base Amount": base,
        "Discount": discount,
        "Tax": tax,
        "Total": total
    }

room_type = input("Enter room type (Standard/Deluxe/Suite): ")
days = int(input("Enter number of days: "))
is_member = input("Are you a member? (yes/no): ").lower() == "yes"

bill = book_room(room_type, days, is_member)
for k, v in bill.items():
    print(f"{k}: {v}")
```

```
Enter room type (Standard/Deluxe/Suite):  deluxe
Enter number of days:  6
Are you a member? (yes/no):  yes
Room Type: deluxe
Days: 6
Base Amount: 0
Discount: 0.0
Tax: 0.0
Total: 0.0
```

Q19. Flight Fare Calculator with International Tax

Function calculate_fare(distance, class_type, international=False):

Economy: 10/unit, Business: 25/unit, First: 40/unit.

Add fuel surcharge 500.

If international → add 18% tax.

If booking in advance (>30 days) → 10% discount.

[11]:
```
def calculate_fare(distance, class_type, international=False, advance_days=0):
    rates = {"Economy": 10, "Business": 25, "First": 40}
    base = rates.get(class_type, 0) * distance
    fuel_surcharge = 500
    subtotal = base + fuel_surcharge
    if international:
        subtotal += 0.18 * subtotal
    if advance_days > 30:
        subtotal -= 0.1 * subtotal
    return {
        "Class": class_type,
        "Distance": distance,
        "Base Fare": base,
        "Fuel Surcharge": fuel_surcharge,
        "Total Fare": subtotal
    }

distance = float(input("Enter distance: "))
class_type = input("Enter class type (Economy/Business/First): ")
international = input("Is it international? (yes/no): ").lower() == "yes"
advance_days = int(input("Days in advance of booking: "))

fare = calculate_fare(distance, class_type, international, advance_days)
for k, v in fare.items():
    print(f"{k}: {v}")
```

```
Enter distance:  45
Enter class type (Economy/Business/First):  first
Is it international? (yes/no):  yes
Days in advance of booking:  5
Class: first
Distance: 45.0
Base Fare: 0.0
Fuel Surcharge: 500
Total Fare: 590.0
```

Q20. Smart Vending Machine with Wallet System

Function vending_machine(item, amount_paid, wallet_balance):

Items stored in dict with stock + price.

If sufficient → dispense + return change.

Update wallet balance.

If stock = 0 → suggest alternative item.

[12]:
```
def vending_machine(item, amount_paid, wallet_balance):
    items = {
        "Chips": {"price": 30, "stock": 5},
        "Soda": {"price": 50, "stock": 3},
        "Chocolate": {"price": 40, "stock": 0}
    }
    if item not in items:
        return "Item not available", wallet_balance
    if items[item]["stock"] == 0:
        alternatives = [i for i in items if items[i]["stock"] > 0]
        return f"{item} out of stock. Try: {', '.join(alternatives)}", wallet_balance
    price = items[item]["price"]
    if amount_paid + wallet_balance < price:
        return "Insufficient funds", wallet_balance
    items[item]["stock"] -= 1
    total_paid = amount_paid + wallet_balance
    change = total_paid - price
    wallet_balance = change
    return f"Dispensed {item}. Change in wallet: {wallet_balance}", wallet_balance

wallet_balance = float(input("Enter wallet balance: "))
item = input("Enter item to buy: ")
amount_paid = float(input("Enter amount paid: "))

message, wallet_balance = vending_machine(item, amount_paid, wallet_balance)
print(message)
print("Updated wallet balance:", wallet_balance)
```

```
Enter wallet balance:  50
Enter item to buy:  7
Enter amount paid:  28
Item not available
Updated wallet balance: 50.0
```

LAB 5

Q21: Create a `BankAccount` class that represents a bank account. It should have attributes for account number, account holder name, and balance. Include methods to deposit, withdraw, and display balance. Ensure withdrawals cannot exceed the available balance.

[13]:
```
class BankAccount:
    def __init__(self, account_number, holder_name, balance=0):
        self.account_number = account_number
        self.holder_name = holder_name
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        return f"Deposited {amount}. New balance: {self.balance}"

    def withdraw(self, amount):
        if amount > self.balance:
            return "Insufficient balance"
        self.balance -= amount
        return f"Withdrew {amount}. New balance: {self.balance}"

    def display_balance(self):
        return f"Account Number: {self.account_number}\nHolder Name: {self.holder_name}\nBalance: {self.balance}"

account_number = input("Enter account number: ")
holder_name = input("Enter account holder name: ")
balance = float(input("Enter initial balance: "))

account = BankAccount(account_number, holder_name, balance)

print(account.deposit(500))
print(account.withdraw(200))
print(account.display_balance())
```

```
Enter account number:  454545
Enter account holder name:  yumna
Enter initial balance:  500
Deposited 500. New balance: 1000.0
Withdrew 200. New balance: 800.0
Account Number: 454545
Holder Name: yumna
Balance: 800.0
```

Q22: Design a `Book` class with attributes like title, author, ISBN, and availability status. Create a `Library` class that can add books, search for books by title/author, check out books, and return books. Track which books are currently available.

[14]:
```python
class Book:
    def __init__(self, title, author, isbn, available=True):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.available = available

class Library:
    def __init__(self):
        self.books = []

    def add_book(self, book):
        self.books.append(book)
        return f"Added book: {book.title}"

    def search_by_title(self, title):
        return [b for b in self.books if title.lower() in b.title.lower()]

    def search_by_author(self, author):
        return [b for b in self.books if author.lower() in b.author.lower()]

    def check_out(self, isbn):
        for b in self.books:
            if b.isbn == isbn:
                if b.available:
                    b.available = False
                    return f"Book '{b.title}' checked out"
                else:
                    return f"Book '{b.title}' is not available"
        return "Book not found"

    def return_book(self, isbn):
        for b in self.books:
            if b.isbn == isbn:
                b.available = True
                return f"Book '{b.title}' returned"
        return "Book not found"

library = Library()
library.add_book(Book("Python Basics", "John Doe", "123"))
library.add_book(Book("Advanced Python", "Jane Smith", "456"))

print([b.title for b in library.search_by_title("python")])
print(library.check_out("123"))
print(library.check_out("123"))
print(library.return_book("123"))
```

```
['Python Basics', 'Advanced Python']
Book 'Python Basics' checked out
Book 'Python Basics' is not available
Book 'Python Basics' returned
```

Q23: Create a `Student` class with attributes for student ID, name, and a list of grades. Implement methods to add grades, calculate average grade, and determine the letter grade (A, B, C, etc.). Add a method to display student information with their performance summary.

[15]:
```
class Student:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name
        self.grades = []

    def add_grade(self, grade):
        self.grades.append(grade)
        return f"Added grade: {grade}"

    def average_grade(self):
        if not self.grades:
            return 0
        return sum(self.grades) / len(self.grades)

    def letter_grade(self):
        avg = self.average_grade()
        if avg >= 90:
            return "A"
        elif avg >= 80:
            return "B"
        elif avg >= 70:
            return "C"
        elif avg >= 60:
            return "D"
        else:
            return "F"

    def display_info(self):
        avg = self.average_grade()
        letter = self.letter_grade()
        return f"Student ID: {self.student_id}\nName: {self.name}\nGrades: {self.grades}\nAverage: {avg}\nLetter Grade: {letter}"

student = Student("S001", "Alice")
student.add_grade(95)
student.add_grade(82)
student.add_grade(76)

print(student.display_info())
```

```
Student ID: S001
Name: Alice
Grades: [95, 82, 76]
Average: 84.33333333333333
Letter Grade: B
```

Q24: Design a `MenuItem` class for food items (name, price, category) and an `Order` class that can add multiple items, calculate total cost, apply discounts, and generate receipts. Include tax calculation functionality.

[16]:
```
class MenuItem:
    def __init__(self, name, price, category):
        self.name = name
        self.price = price
        self.category = category

class Order:
    def __init__(self, tax_rate=0.12):
        self.items = []
        self.tax_rate = tax_rate

    def add_item(self, item, quantity=1):
        self.items.append((item, quantity))
        return f"Added {quantity} x {item.name}"

    def total_cost(self):
        subtotal = sum(item.price * qty for item, qty in self.items)
        return subtotal

    def apply_discount(self, discount_rate):
        subtotal = self.total_cost()
        return subtotal * (1 - discount_rate)

    def generate_receipt(self, discount_rate=0):
        subtotal = self.total_cost()
        discounted = subtotal * (1 - discount_rate)
        tax = discounted * self.tax_rate
        total = discounted + tax
        receipt = "Receipt:\n"
        for item, qty in self.items:
            receipt += f"{item.name} x {qty} = {item.price*qty}\n"
        receipt += f"Subtotal: {subtotal}\nDiscount: {subtotal - discounted}\nTax: {tax}\nTotal: {total}"
        return receipt

burger = MenuItem("Burger", 150, "Main")
fries = MenuItem("Fries", 50, "Snack")

order = Order()
order.add_item(burger, 2)
order.add_item(fries, 3)

print(order.generate_receipt(discount_rate=0.1))
```

```
Receipt:
Burger x 2 = 300
Fries x 3 = 150
Subtotal: 450
Discount: 45.0
Tax: 48.6
Total: 453.6
```

Q25: Create classes for `Vehicle` (base class) and derived classes `Car`, `Motorcycle`, and `Truck`. Each vehicle should have attributes like license plate, model, rental price per day, and availability. Implement a system to rent vehicles and calculate rental costs.

[17]:
```
class Vehicle:
    def __init__(self, license_plate, model, price_per_day, available=True):
        self.license_plate = license_plate
        self.model = model
        self.price_per_day = price_per_day
        self.available = available

    def rent(self, days):
        if not self.available:
            return f"{self.model} not available"
        self.available = False
        return f"Rented {self.model} for {days} days. Total cost: {self.price_per_day * days}"

    def return_vehicle(self):
        self.available = True
        return f"{self.model} returned and now available"

class Car(Vehicle):
    pass

class Motorcycle(Vehicle):
    pass

class Truck(Vehicle):
    pass

vehicles = [
    Car("ABC123", "Toyota Camry", 2000),
    Motorcycle("XYZ789", "Yamaha R15", 800),
    Truck("TRK456", "Volvo Truck", 5000)
]

for v in vehicles:
    print(v.rent(3))

print(vehicles[0].return_vehicle())
print(vehicles[0].rent(2))
```

```
Rented Toyota Camry for 3 days. Total cost: 6000
Rented Yamaha R15 for 3 days. Total cost: 2400
Rented Volvo Truck for 3 days. Total cost: 15000
Toyota Camry returned and now available
Rented Toyota Camry for 2 days. Total cost: 4000
```

Lab 6

Q26: Design an `Employee` class with attributes for employee ID, name, position, and salary. Create methods to calculate monthly salary, annual salary, and apply raises. Include functionality to track employee details and employment duration.

[18]:
```python
from datetime import date

class Employee:
    def __init__(self, emp_id, name, position, salary, join_date):
        self.emp_id = emp_id
        self.name = name
        self.position = position
        self.salary = salary
        self.join_date = join_date

    def monthly_salary(self):
        return self.salary

    def annual_salary(self):
        return self.salary * 12

    def apply_raise(self, percentage):
        self.salary += self.salary * (percentage / 100)
        return f"New salary: {self.salary}"

    def employment_duration(self):
        today = date.today()
        duration = today - self.join_date
        return f"Employment duration: {duration.days} days"

    def display_info(self):
        return f"ID: {self.emp_id}\nName: {self.name}\nPosition: {self.position}\nSalary: {self.salary}\n{self.employment_duration()}"

emp = Employee("E001", "John Doe", "Developer", 5000, date(2020, 5, 1))
print(emp.display_info())
print("Monthly Salary:", emp.monthly_salary())
print("Annual Salary:", emp.annual_salary())
print(emp.apply_raise(10))
```

```
ID: E001
Name: John Doe
Position: Developer
Salary: 5000
Employment duration: 2068 days
Monthly Salary: 5000
Annual Salary: 60000
New salary: 5500.0
```

Q27: Create a `Product` class with attributes for product ID, name, price, and stock quantity. Design a `ShoppingCart` class that can add products, remove products, calculate total cost, and handle checkout process while updating inventory.

[19]:
```python
class Product:
    def __init__(self, product_id, name, price, stock):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.stock = stock

class ShoppingCart:
    def __init__(self):
        self.items = []

    def add_product(self, product, quantity):
        if product.stock < quantity:
            return f"Not enough stock for {product.name}"
        self.items.append((product, quantity))
        product.stock -= quantity
        return f"Added {quantity} x {product.name}"

    def remove_product(self, product_id):
        for i, (product, qty) in enumerate(self.items):
            if product.product_id == product_id:
                product.stock += qty
                del self.items[i]
                return f"Removed {product.name}"
        return "Product not in cart"

    def total_cost(self):
        return sum(product.price * qty for product, qty in self.items)

    def checkout(self):
        total = self.total_cost()
        self.items.clear()
        return f"Checkout complete. Total cost: {total}"

p1 = Product("P001", "Laptop", 50000, 5)
p2 = Product("P002", "Mouse", 500, 10)

cart = ShoppingCart()
print(cart.add_product(p1, 1))
print(cart.add_product(p2, 2))
print("Total cost:", cart.total_cost())
print(cart.checkout())
print("Laptop stock left:", p1.stock)
```

```
Added 1 x Laptop
Added 2 x Mouse
Total cost: 51000
Checkout complete. Total cost: 51000
Laptop stock left: 4
```

Q28: Design a `Patient` class to store patient information (ID, name, age, medical history) and an `Appointment` class to manage doctor appointments. Include functionality to schedule, cancel, and view appointments.

[20]:
```
class Patient:
    def __init__(self, patient_id, name, age, medical_history=None):
        self.patient_id = patient_id
        self.name = name
        self.age = age
        self.medical_history = medical_history or []

    def add_history(self, record):
        self.medical_history.append(record)
        return f"Added record: {record}"

class Appointment:
    def __init__(self):
        self.appointments = {}

    def schedule(self, patient, date_time, doctor):
        key = (date_time, doctor)
        if key in self.appointments:
            return "Slot already booked"
        self.appointments[key] = patient
        return f"Appointment scheduled for {patient.name} with Dr.{doctor} on {date_time}"

    def cancel(self, date_time, doctor):
        key = (date_time, doctor)
        if key in self.appointments:
            patient = self.appointments.pop(key)
            return f"Appointment for {patient.name} canceled"
        return "No appointment found"

    def view_appointments(self):
        return {f"{dt} with Dr.{doc}": pat.name for (dt, doc), pat in self.appointments.items()}

p1 = Patient("PT001", "Alice", 30)
appt = Appointment()
print(appt.schedule(p1, "2025-01-05 10:00", "Smith"))
print(appt.view_appointments())
print(appt.cancel("2025-01-05 10:00", "Smith"))
print(appt.view_appointments())
```

```
Appointment scheduled for Alice with Dr.Smith on 2025-01-05 10:00
{'2025-01-05 10:00 with Dr.Smith': 'Alice'}
Appointment for Alice canceled
{}
```

Q29: Create a `User` class for a social media platform with attributes for username, email, friends list, and posts. Implement methods to add friends, create posts, and display user timeline. Include privacy settings for posts

[21]:
```
class Post:
    def __init__(self, content, privacy="Public"):
        self.content = content
        self.privacy = privacy

class User:
    def __init__(self, username, email):
        self.username = username
        self.email = email
        self.friends = []
        self.posts = []

    def add_friend(self, user):
        if user.username not in [f.username for f in self.friends]:
            self.friends.append(user)
            return f"{user.username} added as friend"
        return f"{user.username} is already a friend"

    def create_post(self, content, privacy="Public"):
        post = Post(content, privacy)
        self.posts.append(post)
        return "Post created"

    def timeline(self, viewer=None):
        visible_posts = []
        for post in self.posts:
            if post.privacy == "Public" or (post.privacy == "Friends" and viewer in self.friends):
                visible_posts.append(post.content)
        return visible_posts

user1 = User("Alice", "alice@example.com")
user2 = User("Bob", "bob@example.com")

print(user1.add_friend(user2))
print(user1.create_post("Hello World!", "Public"))
print(user1.create_post("Secret post", "Friends"))
print("Timeline for Bob:", user1.timeline(viewer=user2))
print("Timeline for Alice:", user1.timeline(viewer=None))
```

```
Bob added as friend
Post created
Post created
Timeline for Bob: ['Hello World!', 'Secret post']
Timeline for Alice: ['Hello World!']
```

Q30: Design a `Room` class with room number, type (single/double/suite), price, and availability. Create a `Booking` class to handle room reservations, check-in/check-out dates, and calculate total stay cost.

[22]:
```
class Room:
    def __init__(self, room_number, room_type, price, available=True):
        self.room_number = room_number
        self.room_type = room_type
        self.price = price
        self.available = available

class Booking:
    def __init__(self):
        self.bookings = []

    def reserve(self, room, check_in, check_out):
        if not room.available:
            return f"Room {room.room_number} not available"
        room.available = False
        self.bookings.append({"room": room, "check_in": check_in, "check_out": check_out})
        days = (check_out - check_in).days
        total = days * room.price
        return f"Room {room.room_number} booked from {check_in} to {check_out}. Total cost: {total}"

    def check_out(self, room):
        room.available = True
        self.bookings = [b for b in self.bookings if b["room"] != room]
        return f"Room {room.room_number} is now available"

from datetime import date

room1 = Room(101, "Single", 2000)
room2 = Room(102, "Double", 3500)

booking_system = Booking()
print(booking_system.reserve(room1, date(2025, 1, 5), date(2025, 1, 8)))
print(booking_system.reserve(room1, date(2025, 1, 10), date(2025, 1, 12)))
print(booking_system.check_out(room1))
print(booking_system.reserve(room1, date(2025, 1, 10), date(2025, 1, 12)))
```

```
Room 101 booked from 2025-01-05 to 2025-01-08. Total cost: 6000
Room 101 not available
Room 101 is now available
Room 101 booked from 2025-01-10 to 2025-01-12. Total cost: 4000
```

Lab 7

Q31: Create `Course` classes (with code, name, credits, capacity) and a `Student` class that can register for courses. Implement functionality to check for schedule conflicts and ensure students don't exceed credit limits.

[23]:
```
class Course:
    def __init__(self, code, name, credits, capacity, schedule):
        self.code = code
        self.name = name
        self.credits = credits
        self.capacity = capacity
        self.schedule = schedule
        self.enrolled_students = []

class Student:
    def __init__(self, student_id, name, max_credits=18):
        self.student_id = student_id
        self.name = name
        self.registered_courses = []
        self.max_credits = max_credits

    def register_course(self, course):
        if len(course.enrolled_students) >= course.capacity:
            return f"{course.name} is full"
        current_credits = sum(c.credits for c in self.registered_courses)
        if current_credits + course.credits > self.max_credits:
            return "Credit limit exceeded"
        for c in self.registered_courses:
            if set(c.schedule).intersection(set(course.schedule)):
                return f"Schedule conflict with {c.name}"
        course.enrolled_students.append(self)
        self.registered_courses.append(course)
        return f"Registered for {course.name}"

c1 = Course("CS101", "Intro to CS", 4, 2, ["Mon 9-11"])
c2 = Course("MA101", "Calculus", 3, 2, ["Mon 10-12"])
c3 = Course("PH101", "Physics", 3, 2, ["Tue 9-11"])

student = Student("S001", "Alice")
print(student.register_course(c1))
print(student.register_course(c2))
print(student.register_course(c3))
```

```
Registered for Intro to CS
Registered for Calculus
Registered for Physics
```

Q32: Design a `Product` class for items in inventory (ID, name, quantity, price, supplier) and an `Inventory` class to track stock levels, add new products, update quantities, and generate low-stock alerts.

[24]:
```python
class Product:
    def __init__(self, product_id, name, quantity, price, supplier):
        self.product_id = product_id
        self.name = name
        self.quantity = quantity
        self.price = price
        self.supplier = supplier

class Inventory:
    def __init__(self):
        self.products = {}

    def add_product(self, product):
        self.products[product.product_id] = product
        return f"Added {product.name}"

    def update_quantity(self, product_id, quantity):
        if product_id in self.products:
            self.products[product_id].quantity += quantity
            return f"Updated {self.products[product_id].name} quantity to {self.products[product_id].quantity}"
        return "Product not found"

    def low_stock_alert(self, threshold):
        return [p.name for p in self.products.values() if p.quantity <= threshold]

p1 = Product("P001", "Laptop", 5, 50000, "SupplierA")
p2 = Product("P002", "Mouse", 2, 500, "SupplierB")

inventory = Inventory()
inventory.add_product(p1)
inventory.add_product(p2)

print(inventory.update_quantity("P002", 3))
print("Low stock products:", inventory.low_stock_alert(3))
```

```
Updated Mouse quantity to 5
Low stock products: []
```

Q33: Create `Movie` classes (title, genre, duration, showtimes) and a `Theater` class with seating arrangement. Implement a booking system that reserves seats and calculates ticket prices with different categories (standard, premium).

[25]:
```
class Movie:
    def __init__(self, title, genre, duration, showtimes):
        self.title = title
        self.genre = genre
        self.duration = duration
        self.showtimes = showtimes

class Theater:
    def __init__(self, rows, cols):
        self.seats = [["Available" for _ in range(cols)] for _ in range(rows)]
        self.rows = rows
        self.cols = cols

    def display_seats(self):
        for r in range(self.rows):
            print(f"Row {r+1}: {self.seats[r]}")

    def book_seat(self, row, col, category="Standard"):
        if self.seats[row][col] != "Available":
            return "Seat already booked"
        price = 100 if category == "Standard" else 200
        self.seats[row][col] = "Booked"
        return f"Seat booked. Price: {price}"

movie = Movie("Inception", "Sci-Fi", 150, ["10:00", "14:00", "18:00"])
theater = Theater(5, 5)

theater.display_seats()
print(theater.book_seat(0, 0, "Premium"))
theater.display_seats()
```

```
Row 1: ['Available', 'Available', 'Available', 'Available', 'Available']
Row 2: ['Available', 'Available', 'Available', 'Available', 'Available']
Row 3: ['Available', 'Available', 'Available', 'Available', 'Available']
Row 4: ['Available', 'Available', 'Available', 'Available', 'Available']
Row 5: ['Available', 'Available', 'Available', 'Available', 'Available']
Seat booked. Price: 200
Row 1: ['Booked', 'Available', 'Available', 'Available', 'Available']
Row 2: ['Available', 'Available', 'Available', 'Available', 'Available']
Row 3: ['Available', 'Available', 'Available', 'Available', 'Available']
Row 4: ['Available', 'Available', 'Available', 'Available', 'Available']
Row 5: ['Available', 'Available', 'Available', 'Available', 'Available']
```

Q34: Design a `Member` class with personal details, membership type (basic/premium), and attendance records. Create a `Trainer` class and a system to schedule training sessions between members and trainers.

[26]:
```python
class Member:
    def __init__(self, member_id, name, membership_type):
        self.member_id = member_id
        self.name = name
        self.membership_type = membership_type
        self.attendance = []

    def record_attendance(self, date):
        self.attendance.append(date)
        return f"Attendance recorded for {date}"

class Trainer:
    def __init__(self, trainer_id, name, specialty):
        self.trainer_id = trainer_id
        self.name = name
        self.specialty = specialty
        self.schedule = []

class TrainingSystem:
    def __init__(self):
        self.sessions = []

    def schedule_session(self, member, trainer, date_time):
        for s in self.sessions:
            if s["trainer"] == trainer and s["date_time"] == date_time:
                return "Trainer not available at this time"
        self.sessions.append({"member": member, "trainer": trainer, "date_time": date_time})
        return f"Session scheduled for {member.name} with {trainer.name} on {date_time}"

member1 = Member("M001", "Alice", "Premium")
trainer1 = Trainer("T001", "Bob", "Yoga")
system = TrainingSystem()

print(member1.record_attendance("2025-01-05"))
print(system.schedule_session(member1, trainer1, "2025-01-10 10:00"))
```

```
Attendance recorded for 2025-01-05
Session scheduled for Alice with Bob on 2025-01-10 10:00
```

Q35: Extend the bank account system to include `SavingsAccount` and `CheckingAccount` classes with different interest rates and withdrawal rules. Implement fund transfer between accounts.

[27]:
```python
class BankAccount:
    def __init__(self, account_number, holder_name, balance=0):
        self.account_number = account_number
        self.holder_name = holder_name
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        return f"Deposited {amount}. New balance: {self.balance}"

    def withdraw(self, amount):
        if amount > self.balance:
            return "Insufficient balance"
        self.balance -= amount
        return f"Withdrew {amount}. New balance: {self.balance}"

    def display_balance(self):
        return f"Account Number: {self.account_number}\nHolder Name: {self.holder_name}\nBalance: {self.balance}"

    def transfer(self, target_account, amount):
        if amount > self.balance:
            return "Insufficient balance to transfer"
        self.balance -= amount
        target_account.balance += amount
        return f"Transferred {amount} to {target_account.holder_name}. New balance: {self.balance}"

class SavingsAccount(BankAccount):
    def __init__(self, account_number, holder_name, balance=0, interest_rate=0.04):
        super().__init__(account_number, holder_name, balance)
        self.interest_rate = interest_rate

    def apply_interest(self):
        self.balance += self.balance * self.interest_rate
        return f"Interest applied. New balance: {self.balance}"

class CheckingAccount(BankAccount):
    def __init__(self, account_number, holder_name, balance=0, overdraft_limit=1000):
        super().__init__(account_number, holder_name, balance)
        self.overdraft_limit = overdraft_limit

    def withdraw(self, amount):
        if amount > self.balance + self.overdraft_limit:
            return "Exceeds overdraft limit"
        self.balance -= amount
        return f"Withdrew {amount}. New balance: {self.balance}"

acc1 = SavingsAccount("S001", "Alice", 10000)
acc2 = CheckingAccount("C001", "Bob", 5000)

print(acc1.deposit(2000))
print(acc1.apply_interest())
print(acc1.transfer(acc2, 3000))
print(acc2.withdraw(7000))
print(acc1.display_balance())
print(acc2.display_balance())
```

```
Deposited 2000. New balance: 12000
Interest applied. New balance: 12480.0
Transferred 3000 to Bob. New balance: 9480.0
Withdrew 7000. New balance: 1000
Account Number: S001
Holder Name: Alice
Balance: 9480.0
Account Number: C001
Holder Name: Bob
Balance: 1000
```

Q36: Create a `Pizza` class with size, crust type, and toppings. Design an ordering system that calculates prices based on selections and allows custom pizza creation with various topping combinations.

[28]:
```
class Pizza:
    base_prices = {"Small": 200, "Medium": 300, "Large": 400}
    crust_prices = {"Thin": 50, "Thick": 70, "Cheese Stuffed": 100}
    topping_prices = {"Pepperoni": 50, "Mushroom": 30, "Onion": 20, "Capsicum": 25, "Extra Cheese": 40}

    def __init__(self, size, crust, toppings=None):
        self.size = size
        self.crust = crust
        self.toppings = toppings or []

    def calculate_price(self):
        price = self.base_prices.get(self.size, 0) + self.crust_prices.get(self.crust, 0)
        price += sum(self.topping_prices.get(t, 0) for t in self.toppings)
        return price

    def display_pizza(self):
        return f"Size: {self.size}\nCrust: {self.crust}\nToppings: {', '.join(self.toppings)}\nTotal Price: {self.calculate_price()}"

pizza1 = Pizza("Medium", "Thin", ["Pepperoni", "Mushroom", "Extra Cheese"])
print(pizza1.display_pizza())

pizza2 = Pizza("Large", "Cheese Stuffed", ["Onion", "Capsicum"])
print(pizza2.display_pizza())
```

```
Size: Medium
Crust: Thin
Toppings: Pepperoni, Mushroom, Extra Cheese
Total Price: 470
Size: Large
Crust: Cheese Stuffed
Toppings: Onion, Capsicum
Total Price: 545
```

Q37: Design classes for `Car` (model, year, color, price) and `CarManufacturer` that can produce different car models with various features. Include quality control checks and production tracking.

[29]:
```
class Car:
    def __init__(self, model, year, color, price):
        self.model = model
        self.year = year
        self.color = color
        self.price = price
        self.passed_quality_check = False

    def quality_check(self):
        self.passed_quality_check = True
        return f"{self.model} passed quality check"

class CarManufacturer:
    def __init__(self, name):
        self.name = name
        self.production_log = []

    def produce_car(self, model, year, color, price):
        car = Car(model, year, color, price)
        car.quality_check()
        self.production_log.append(car)
        return f"Produced {car.model} ({car.color}, {car.year})"

    def production_summary(self):
        return [f"{c.model} ({c.color}, {c.year}) - Quality Passed: {c.passed_quality_check}" for c in self.production_log]

manufacturer = CarManufacturer("AutoMakers Inc.")
print(manufacturer.produce_car("Sedan X", 2025, "Red", 20000))
print(manufacturer.produce_car("SUV Y", 2025, "Black", 35000))
print("Production Summary:", manufacturer.production_summary())
```

```
Produced Sedan X (Red, 2025)
Produced SUV Y (Black, 2025)
Production Summary: ['Sedan X (Red, 2025) - Quality Passed: True', 'SUV Y (Black, 2025) - Quality Passed: True']
```

\

Lab 8

Q38: Create `Teacher` and `Student` classes, and a `Classroom` class that manages assignments, grades, and attendance. Implement functionality to track student performance and generate class reports.

[30]:
```python
class Student:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name
        self.grades = {}
        self.attendance = []

    def add_grade(self, assignment, grade):
        self.grades[assignment] = grade

    def record_attendance(self, date):
        self.attendance.append(date)

    def average_grade(self):
        if not self.grades:
            return 0
        return sum(self.grades.values()) / len(self.grades)

class Teacher:
    def __init__(self, teacher_id, name):
        self.teacher_id = teacher_id
        self.name = name

class Classroom:
    def __init__(self, class_name, teacher):
        self.class_name = class_name
        self.teacher = teacher
        self.students = []

    def add_student(self, student):
        self.students.append(student)

    def generate_report(self):
        report = {}
        for student in self.students:
            report[student.name] = {
                "Average Grade": student.average_grade(),
                "Attendance": len(student.attendance)
            }
        return report

teacher = Teacher("T001", "Mr. Smith")
classroom = Classroom("Math101", teacher)
s1 = Student("S001", "Alice")
s2 = Student("S002", "Bob")
classroom.add_student(s1)
classroom.add_student(s2)

s1.add_grade("Assignment1", 85)
s1.add_grade("Assignment2", 90)
s1.record_attendance("2025-01-05")
s2.add_grade("Assignment1", 75)
s2.record_attendance("2025-01-05")

print(classroom.generate_report())
```

```
{'Alice': {'Average Grade': 87.5, 'Attendance': 1}, 'Bob': {'Average Grade': 75.0, 'Attendance': 1}}
```

Q39: Design `Flight` classes (flight number, destination, departure time, capacity) and a `Passenger` class. Create a booking system that reserves seats and manages passenger manifests.

[31]:
```python
class Passenger:
    def __init__(self, passenger_id, name):
        self.passenger_id = passenger_id
        self.name = name

class Flight:
    def __init__(self, flight_number, destination, departure_time, capacity):
        self.flight_number = flight_number
        self.destination = destination
        self.departure_time = departure_time
        self.capacity = capacity
        self.passengers = []

    def book_seat(self, passenger):
        if len(self.passengers) >= self.capacity:
            return f"Flight {self.flight_number} is full"
        self.passengers.append(passenger)
        return f"Seat booked for {passenger.name} on flight {self.flight_number}"

    def passenger_list(self):
        return [p.name for p in self.passengers]

flight = Flight("FL123", "New York", "2025-01-10 08:00", 2)
p1 = Passenger("P001", "Alice")
p2 = Passenger("P002", "Bob")
p3 = Passenger("P003", "Charlie")

print(flight.book_seat(p1))
print(flight.book_seat(p2))
print(flight.book_seat(p3))
print("Passengers on flight:", flight.passenger_list())
```

```
Seat booked for Alice on flight FL123
Seat booked for Bob on flight FL123
Flight FL123 is full
Passengers on flight: ['Alice', 'Bob']
```

Q40: Create classes for `SmartDevice` (base class) and specific devices like `SmartLight`, `Thermostat`, and `SecurityCamera`. Implement a `SmartHome` class that can control all devices and create automation routines.

[32]:
```python
class SmartDevice:
    def __init__(self, name):
        self.name = name
        self.status = "Off"

    def turn_on(self):
        self.status = "On"
        return f"{self.name} turned on"

    def turn_off(self):
        self.status = "Off"
        return f"{self.name} turned off"

class SmartLight(SmartDevice):
    def set_brightness(self, level):
        return f"{self.name} brightness set to {level}%"

class Thermostat(SmartDevice):
    def set_temperature(self, temp):
        return f"{self.name} temperature set to {temp}°C"

class SecurityCamera(SmartDevice):
    def start_recording(self):
        return f"{self.name} started recording"

    def stop_recording(self):
        return f"{self.name} stopped recording"

class SmartHome:
    def __init__(self):
        self.devices = []

    def add_device(self, device):
        self.devices.append(device)
        return f"{device.name} added to SmartHome"

    def control_device(self, device_name, action, *args):
        for device in self.devices:
            if device.name == device_name:
                method = getattr(device, action, None)
                if method:
                    return method(*args)
                return "Action not found"
        return "Device not found"

    def automation_routine(self):
        results = []
        for device in self.devices:
            results.append(device.turn_on())
            if isinstance(device, Thermostat):
                results.append(device.set_temperature(22))
            if isinstance(device, SmartLight):
                results.append(device.set_brightness(75))
        return results
```

```
Living Room Light turned on
Main Thermostat temperature set to 24°C
['Living Room Light turned on', 'Living Room Light brightness set to 75%', 'Main Thermostat turned on', 'Main Thermostat temperature set to 22°C', 'Fron
t Door Camera turned on']
```

Lab 9

Q41: A weather station records temperatures (°C) for 7 days:
[21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0]
Task:

- Convert the list into a NumPy array
- Find:
    - Average temperature
    - Maximum and minimum temperature
    - Temperature difference (range)
    -

```
[33]: import numpy as np

temperatures = [21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0]
temp_array = np.array(temperatures)

average_temp = np.mean(temp_array)
max_temp = np.max(temp_array)
min_temp = np.min(temp_array)
temp_range = max_temp - min_temp

print("Average Temperature:", average_temp)
print("Maximum Temperature:", max_temp)
print("Minimum Temperature:", min_temp)
print("Temperature Range:", temp_range)
```

```
Average Temperature: 23.12857142857143
Maximum Temperature: 26.5
Minimum Temperature: 19.8
Temperature Range: 6.699999999999999
```

Q42: A fruit store records daily sales of apples for 4 weeks:
Week 1: [12, 15, 14, 10, 9, 8, 7]
Week 2: [11, 12, 13, 9, 5, 8, 6]
Week 3: [7, 9, 12, 10, 11, 13, 12]
Week 4: [10, 11, 12, 7, 8, 9, 11]
Task:

- Create a **4×7 NumPy array**
- Find weekly totals
- Find the highest sale day (index only)
- Find the week with the most sales

```
[34]: import numpy as np

week1 = [12, 15, 14, 10, 9, 8, 7]
week2 = [11, 12, 13, 9, 5, 8, 6]
week3 = [7, 9, 12, 10, 11, 13, 12]
week4 = [10, 11, 12, 7, 8, 9, 11]

sales = np.array([week1, week2, week3, week4])

weekly_totals = np.sum(sales, axis=1)
highest_sale_index = np.unravel_index(np.argmax(sales), sales.shape)
week_most_sales = np.argmax(weekly_totals)

print("Weekly Totals:", weekly_totals)
print("Highest Sale Day Index (week, day):", highest_sale_index)
print("Week with Most Sales (0-based index):", week_most_sales)
```

```
Weekly Totals: [75 64 74 68]
Highest Sale Day Index (week, day): (np.int64(0), np.int64(1))
Week with Most Sales (0-based index): 0
```

Q43: Students received marks in a test:
[56, 78, 45, 90, 88, 76, 59, 62]
Task:

- Convert to NumPy array
- Add **5 bonus marks** to every student using broadcasting
- Count how many students now scored > 60
- Find the median and standard deviation
-

[35]:
```
import numpy as np

marks = [56, 78, 45, 90, 88, 76, 59, 62]
marks_array = np.array(marks)

marks_array += 5  # Add 5 bonus marks using broadcasting

count_above_60 = np.sum(marks_array > 60)
median_marks = np.median(marks_array)
std_dev = np.std(marks_array)

print("Marks after bonus:", marks_array)
print("Number of students scoring > 60:", count_above_60)
print("Median:", median_marks)
print("Standard Deviation:", std_dev)
```

```
Marks after bonus: [61 83 50 95 93 81 64 67]
Number of students scoring > 60: 7
Median: 74.0
Standard Deviation: 15.105876340020794
```

Q44: A smartwatch stores daily step-counts for a user for 30 days.
Task:

- Generate a random NumPy array of shape (30,) with values between 2000–15000
- Reshape into a **5×6** matrix
- Find:
    - Weekly average steps
    - Day with maximum steps
    - Number of days user crossed 10,000 steps

[36]:
```
import numpy as np

np.random.seed(0)
steps = np.random.randint(2000, 15001, size=30)

steps_matrix = steps.reshape(5, 6)

weekly_avg = np.mean(steps_matrix, axis=1)
max_step_index = np.unravel_index(np.argmax(steps_matrix), steps_matrix.shape)
days_above_10000 = np.sum(steps_matrix > 10000)

print("Steps Matrix:\n", steps_matrix)
print("Weekly Average Steps:", weekly_avg)
print("Day with Maximum Steps (week, day):", max_step_index)
print("Number of days > 10,000 steps:", days_above_10000)
```

```
Steps Matrix:
 [[ 4732 12799 11845  5264  6859 11225]
 [ 9891  6373  7874  8744  5468  2705]
 [ 4599 12327  4222  9768  4897 11893]
 [ 2537 13085  8216  8921  8036  4163]
 [ 7072  6851  9877  4046  3871  9599]]
Weekly Average Steps: [8787.33333333 6842.5        7951.         7493.         6886.        ]
Day with Maximum Steps (week, day): (np.int64(3), np.int64(1))
Number of days > 10,000 steps: 6
```

Q45: Heart rate readings (in BPM) for 10 patients recorded 4 times per day:

- Use array shape = **(10, 4)**

Task:

- Create an integer array using np.random.randint(60,150,(10,4))
- For each patient:
  - Compute daily average
  - Identify abnormal readings > 120
- For all patients:
  - Compute overall maximum and minimum

[37]:
```
import numpy as np

np.random.seed(0)  # For reproducibility
heart_rates = np.random.randint(60, 150, (10, 4))

daily_avg = np.mean(heart_rates, axis=1)
abnormal_readings = heart_rates > 120
overall_max = np.max(heart_rates)
overall_min = np.min(heart_rates)

print("Heart Rate Readings:\n", heart_rates)
for i, avg in enumerate(daily_avg):
    print(f"Patient {i+1} - Daily Average: {avg}, Abnormal Readings (>120): {heart_rates[i][abnormal_readings[i]]}")

print("Overall Maximum Heart Rate:", overall_max)
print("Overall Minimum Heart Rate:", overall_min)
```

```
Heart Rate Readings:
 [[104 107 124 127]
 [127  69 143  81]
 [ 96 147 130 148]
 [148  72 118 125]
 [ 99 147 106 148]
 [141  97  85 137]
 [132  69  80 140]
 [129 139 107 124]
 [142 148 109  89]
 [ 79  79  74  99]]
Patient 1 - Daily Average: 115.5, Abnormal Readings (>120): [124 127]
Patient 2 - Daily Average: 105.0, Abnormal Readings (>120): [127 143]
Patient 3 - Daily Average: 130.25, Abnormal Readings (>120): [147 130 148]
Patient 4 - Daily Average: 115.75, Abnormal Readings (>120): [148 125]
Patient 5 - Daily Average: 125.0, Abnormal Readings (>120): [147 148]
Patient 6 - Daily Average: 115.0, Abnormal Readings (>120): [141 137]
Patient 7 - Daily Average: 105.25, Abnormal Readings (>120): [132 140]
Patient 8 - Daily Average: 124.75, Abnormal Readings (>120): [129 139 124]
Patient 9 - Daily Average: 122.0, Abnormal Readings (>120): [142 148]
Patient 10 - Daily Average: 82.75, Abnormal Readings (>120): []
Overall Maximum Heart Rate: 148
Overall Minimum Heart Rate: 69
```

Lab 10

Q46: Two branches of a store record monthly sales (in thousands):
Branch A: [122, 135, 150, 160, 155, 140]
Branch B: [110, 130, 148, 165, 158, 145]
Task:

- Compute difference between branches
- Calculate:
  - Average monthly sales of each branch
  - Which branch performed better overall?
- Compute **correlation** between A and B

[38]:
```
import numpy as np

branch_a = np.array([122, 135, 150, 160, 155, 140])
branch_b = np.array([110, 130, 148, 165, 158, 145])

difference = branch_a - branch_b
avg_a = np.mean(branch_a)
avg_b = np.mean(branch_b)
better_branch = "A" if avg_a > avg_b else "B"
correlation = np.corrcoef(branch_a, branch_b)[0, 1]

print("Difference between branches:", difference)
print("Average Sales - Branch A:", avg_a)
print("Average Sales - Branch B:", avg_b)
print("Better Performing Branch:", better_branch)
print("Correlation between A and B:", correlation)
```

```
Difference between branches: [12  5  2 -5 -3 -5]
Average Sales - Branch A: 143.66666666666666
Average Sales - Branch B: 142.66666666666666
Better Performing Branch: A
Correlation between A and B: 0.9811677353198398
```

Q47: Given a 4×4 grayscale image:
[[100, 120, 130, 90],
[80 , 110, 140, 100],
[70 , 90 , 120, 130],
[110, 140, 150, 160]]
Task:

- Convert to NumPy array
- Normalize pixel values (0–1)
- Apply a transformation: new_pixel = pixel * 1.2
- Clip all values > 255
- Compute average brightness before & after transformation

[39]:
```
import numpy as np

image = np.array([
    [100, 120, 130, 90],
    [80 , 110, 140, 100],
    [70 , 90 , 120, 130],
    [110, 140, 150, 160]
], dtype=float)

avg_before = np.mean(image)

normalized = image / 255.0

transformed = image * 1.2

transformed_clipped = np.clip(transformed, 0, 255)

avg_after = np.mean(transformed_clipped)

print("Normalized Image:\n", normalized)
print("Transformed & Clipped Image:\n", transformed_clipped)
print("Average Brightness Before:", avg_before)
print("Average Brightness After:", avg_after)
```

```
Normalized Image:
 [[0.39215686 0.47058824 0.50980392 0.35294118]
 [0.31372549 0.43137255 0.54901961 0.39215686]
 [0.2745098  0.35294118 0.47058824 0.50980392]
 [0.43137255 0.54901961 0.58823529 0.62745098]]
Transformed & Clipped Image:
 [[120. 144. 156. 108.]
 [ 96. 132. 168. 120.]
 [ 84. 108. 144. 156.]
 [132. 168. 180. 192.]]
Average Brightness Before: 115.0
Average Brightness After: 138.0
```

Q48: Dataset contains 100 samples with 4 features each.
Task:

- Generate a **100×4** matrix with values 1–100
- Standardize the dataset using:
- x_scaled = (x - mean) / std
- Perform:
    - Column-wise mean (should be ≈ 0)
    - Column-wise variance (should be ≈ 1)

Verify results using NumPy functions

[40]:
```
import numpy as np

np.random.seed(0)
data = np.random.randint(1, 101, (100, 4))

mean_col = np.mean(data, axis=0)
std_col = np.std(data, axis=0)
data_scaled = (data - mean_col) / std_col

col_mean_after = np.mean(data_scaled, axis=0)
col_var_after = np.var(data_scaled, axis=0)

print("Column-wise mean after scaling:", col_mean_after)
print("Column-wise variance after scaling:", col_var_after)
```

```
Column-wise mean after scaling: [-2.44249065e-17  4.32986980e-17 -1.06026299e-16 -4.77395901e-17]
Column-wise variance after scaling: [1. 1. 1. 1.]
```

Q49: Simulate traffic density (cars per minute) recorded by 6 sensors over 24 hours.
Task:

- Generate array shape (24, 6)
- Find hourly average
- Identify the **busiest sensor** (highest total density)
- Apply a threshold:
    - Replace all values above 50 with 50 (traffic cap)
- Compute energy usage:

energy = Σ(sensor_density × 0.5)

[41]:
```
import numpy as np

np.random.seed(0)
traffic = np.random.randint(0, 101, (24, 6))

hourly_avg = np.mean(traffic, axis=1)
total_per_sensor = np.sum(traffic, axis=0)
busiest_sensor = np.argmax(total_per_sensor)

traffic_capped = np.clip(traffic, 0, 50)
energy_usage = np.sum(traffic_capped * 0.5)

print("Hourly Average Traffic:", hourly_avg)
print("Busiest Sensor (0-based index):", busiest_sensor)
print("Traffic after capping:\n", traffic_capped)
print("Total Energy Usage:", energy_usage)
```

```
Hourly Average Traffic: [49.66666667 64.16666667 58.16666667 59.         54.83333333 71.5
 25.33333333 44.66666667 41.66666667 22.         38.5        35.
 55.33333333 55.66666667 50.         53.66666667 49.5        62.16666667
 43.66666667 55.66666667 34.5        58.33333333 65.83333333 42.5       ]
Busiest Sensor (0-based index): 5
Traffic after capping:
 [[44 47 50 50 50  9]
 [50 21 36 50 50 50]
 [50 12 50 50 39 50]
 [46 50 50 37 25 50]
 [50  9 20 50 50 50]
 [47 50 50 50 50 49]
 [29 19 19 14 39 32]
 [50  9 50 32 31 50]
 [23 35 50 50 28 34]
 [ 0  0 36 50  5 38]
 [17 50  4 42 50 31]
 [ 1 50 41 50 35 11]
 [46 50 50  0 14 50]
 [50 12 42 50 50 50]
 [ 6 50 47  3 50 50]
 [50 50 15 20 50 50]
 [23 50 13 50 48 49]
 [50 41 35 50 50 50]
 [50  0 50 36 34 48]
 [50  3 50 42 50 21]
 [50  0 10 43 50 23]
 [50  2 50 50 35 50]
 [50 50 46 50 20 50]
 [50 27 14 41 50 50]]
Total Energy Usage: 2702.5
```

Q50: You are simulating forces in a mechanical system.
Force vectors applied to 5 components:
F = [[5,2,1],
[3,1,4],
[6,3,2],
[4,2,5],
[7,1,3]]
Transformation matrix:
T = [[2,1,0],
[1,3,1],
[0,2,2]]
Task:

- Compute transformed forces: F_new = F × T
- Compute magnitude of each force vector
- Identify the component experiencing highest transformed force
- Perform eigenvalue decomposition of T

[42]:
```
import numpy as np

F = np.array([[5,2,1],
              [3,1,4],
              [6,3,2],
              [4,2,5],
              [7,1,3]])

T = np.array([[2,1,0],
              [1,3,1],
              [0,2,2]])

F_new = F.dot(T)
magnitudes = np.linalg.norm(F_new, axis=1)
highest_force_component = np.argmax(magnitudes)
eigenvalues, eigenvectors = np.linalg.eig(T)

print("Transformed Forces:\n", F_new)
print("Force Magnitudes:", magnitudes)
print("Component with Highest Force (0-based index):", highest_force_component)
print("Eigenvalues of T:", eigenvalues)
print("Eigenvectors of T:\n", eigenvectors)
```

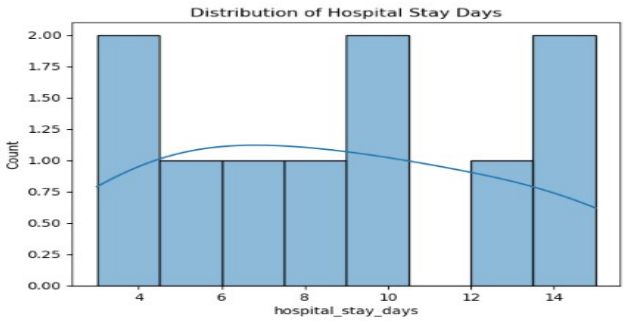
```
Transformed Forces:
 [[12 13  4]
 [ 7 14  9]
 [15 19  7]
 [10 20 12]
 [15 16  7]]
Force Magnitudes: [18.13835715 18.05547009 25.19920634 25.37715508 23.02172887]
Component with Highest Force (0-based index): 3
Eigenvalues of T: [4.30277564 2.         0.69722436]
Eigenvectors of T:
 [[-3.11546502e-01  7.07106781e-01  3.86413754e-01]
 [-7.17421694e-01  2.81676339e-16 -5.03410424e-01]
 [-6.23093003e-01 -7.07106781e-01  7.72827507e-01]]
```

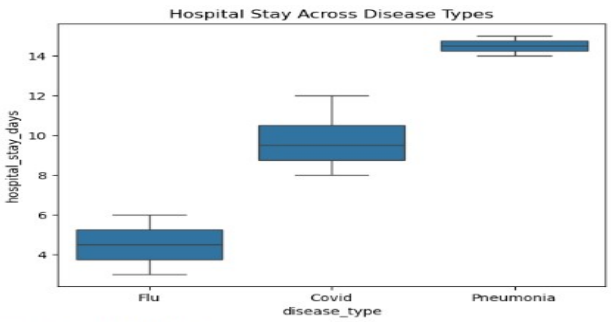
Lab 11

Q51. Patient Stay Analysis (Healthcare)
You are given a dataset containing age, disease_type, and hospital_stay_days.
Write a Python program using **Seaborn** to:

- Plot the distribution of hospital stay days
- Compare hospital stay across different disease types
- Identify which disease has the highest median stay

```
[43]: import pandas as pd
      import seaborn as sns
      import matplotlib.pyplot as plt

      data = pd.DataFrame({
          "age": [25, 34, 45, 50, 60, 38, 42, 55, 48, 30],
          "disease_type": ["Flu", "Covid", "Covid", "Flu", "Pneumonia", "Flu", "Covid", "Pneumonia", "Flu", "Covid"],
          "hospital_stay_days": [3, 10, 12, 5, 15, 4, 8, 14, 6, 9]
      })

      sns.histplot(data['hospital_stay_days'], bins=8, kde=True)
      plt.title("Distribution of Hospital Stay Days")
      plt.show()

      sns.boxplot(x='disease_type', y='hospital_stay_days', data=data)
      plt.title("Hospital Stay Across Disease Types")
      plt.show()

      median_stay = data.groupby('disease_type')['hospital_stay_days'].median()
      disease_highest_median = median_stay.idxmax()

      print("Median Stay per Disease:\n", median_stay)
      print("Disease with Highest Median Stay:", disease_highest_median)
```






```
Median Stay per Disease:
 disease_type
Covid          9.5
Flu            4.5
Pneumonia     14.5
Name: hospital_stay_days, dtype: float64
Disease with Highest Median Stay: Pneumonia
```

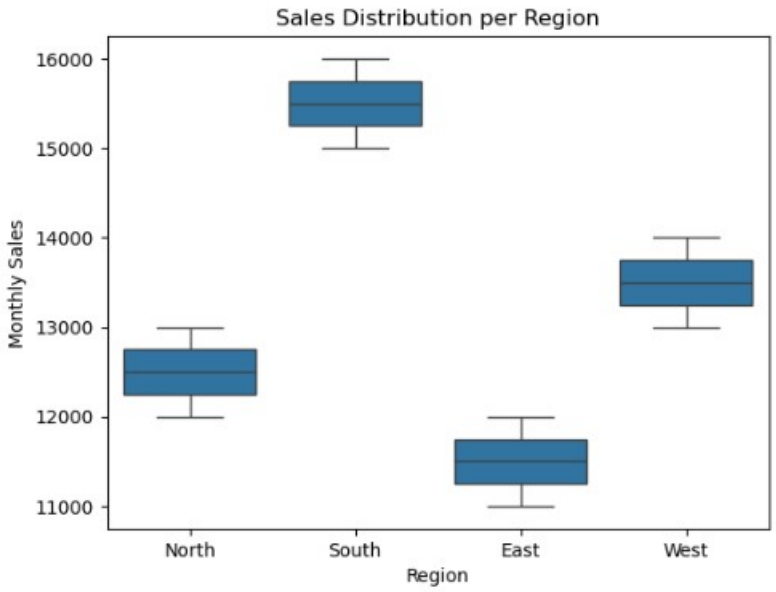
Q52. Sales Distribution Comparison (Business)

A retail dataset contains region and monthly_sales.

Write code to:

- Visualize sales distribution per region using Seaborn
- Detect regions with high sales variability
- Label the axes and add a title

```
[44]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = pd.DataFrame({
    "region": ["North","South","East","West","North","South","East","West","North","South","East","West"],
    "monthly_sales": [12000,15000,11000,13000,12500,16000,11500,13500,13000,15500,12000,14000]
})

sns.boxplot(x='region', y='monthly_sales', data=data)
plt.xlabel("Region")
plt.ylabel("Monthly Sales")
plt.title("Sales Distribution per Region")
plt.show()

sales_variability = data.groupby('region')['monthly_sales'].std()
high_variability_regions = sales_variability[sales_variability > sales_variability.mean()].index.tolist()

print("Sales Standard Deviation per Region:\n", sales_variability)
print("Regions with High Sales Variability:", high_variability_regions)
```




```
Sales Standard Deviation per Region:
 region
East     500.0
North    500.0
South    500.0
West     500.0
Name: monthly_sales, dtype: float64
Regions with High Sales Variability: []
```

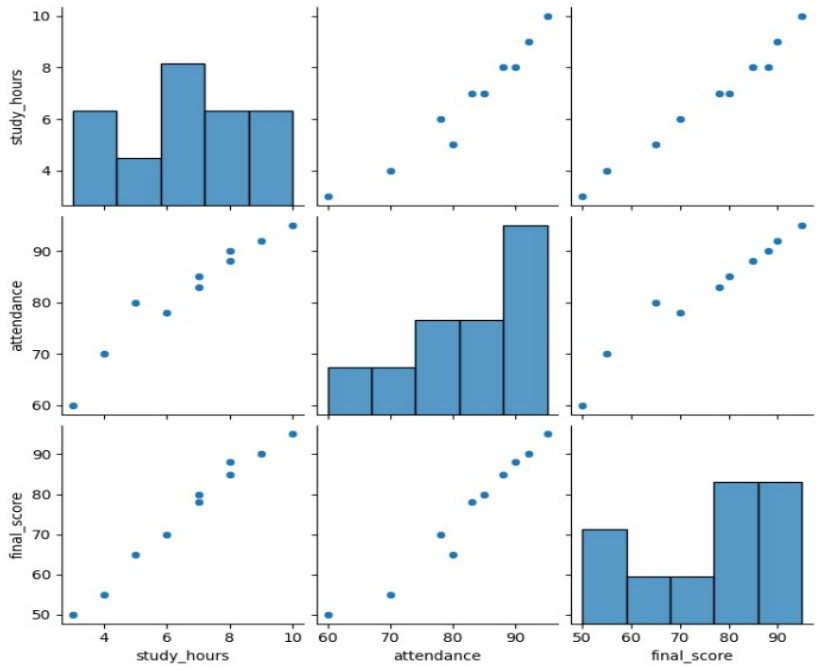
Q53. Student Performance Correlation (Education)

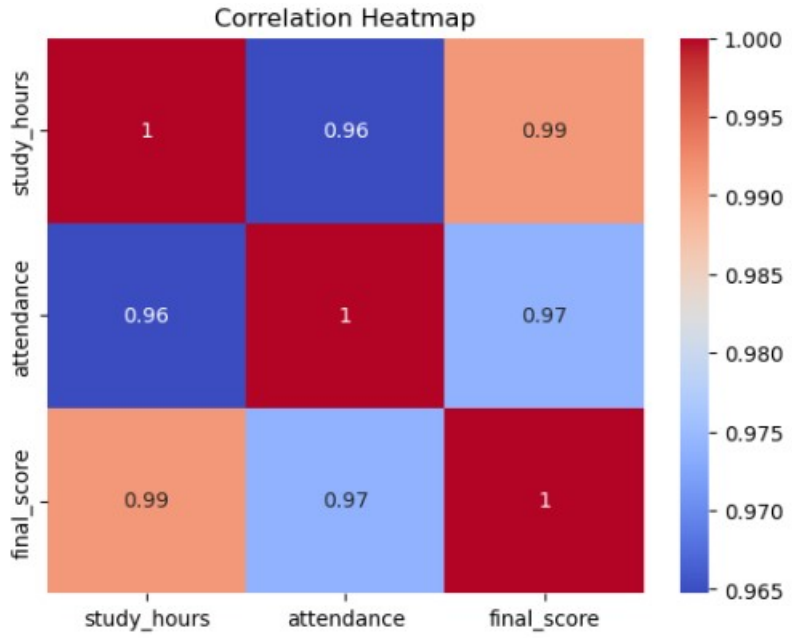
A dataset contains study_hours, attendance, and final_score.

Using Seaborn:

- Create a pair plot
- Plot a correlation heatmap
- Interpret which feature most affects the final score
-

```
[45]: import pandas as pd
      import seaborn as sns
      import matplotlib.pyplot as plt

      data = pd.DataFrame({
          "study_hours": [5, 8, 3, 10, 7, 6, 9, 4, 8, 7],
          "attendance": [80, 90, 60, 95, 85, 78, 92, 70, 88, 83],
          "final_score": [65, 88, 50, 95, 80, 70, 90, 55, 85, 78]
      })

      sns.pairplot(data)
      plt.show()

      corr_matrix = data.corr()
      sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
      plt.title("Correlation Heatmap")
      plt.show()

      most_influential_feature = corr_matrix['final_score'].drop('final_score').idxmax()
      print("Feature most affecting final score:", most_influential_feature)
```






```
Feature most affecting final score: study_hours
```

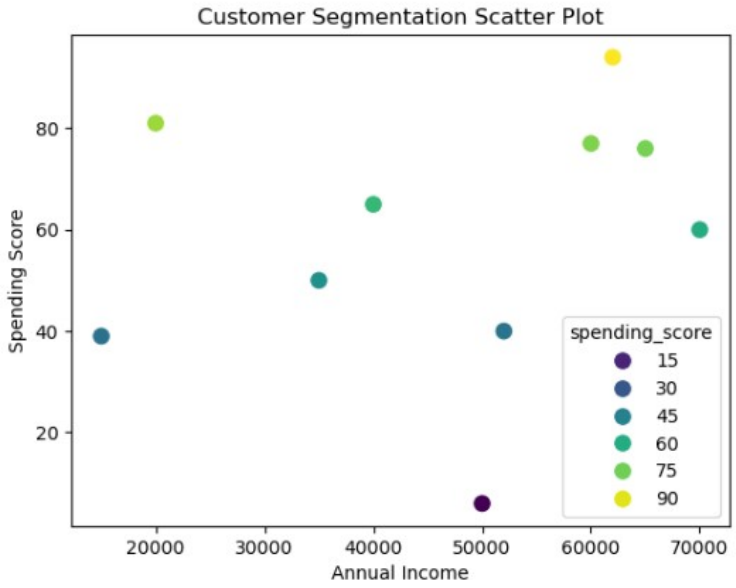
Q54. Customer Segmentation Visualization

Customer data includes age, annual_income, and spending_score.

Write a program to:

- Visualize relationships using Seaborn scatter plots
- Color-code points based on spending score
- Identify potential customer segments visually

```
[46]: import pandas as pd
      import seaborn as sns
      import matplotlib.pyplot as plt

      data = pd.DataFrame({
          "age": [22, 25, 47, 52, 46, 56, 55, 60, 35, 40],
          "annual_income": [15000, 20000, 50000, 60000, 52000, 65000, 62000, 70000, 35000, 40000],
          "spending_score": [39, 81, 6, 77, 40, 76, 94, 60, 50, 65]
      })

      sns.scatterplot(x='annual_income', y='spending_score', hue='spending_score', palette='viridis', data=data, s=100)
      plt.xlabel("Annual Income")
      plt.ylabel("Spending Score")
      plt.title("Customer Segmentation Scatter Plot")
      plt.show()
```



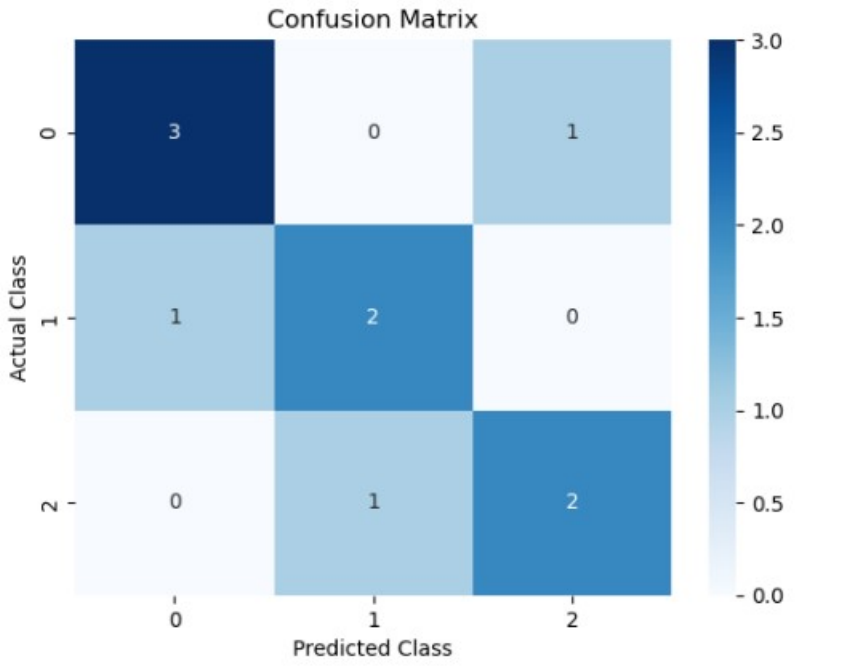
Q55. Model Result Visualization

You trained a classifier and obtained y_true and y_pred.

Write code using Seaborn to:

- Plot a confusion matrix heatmap
- Add annotations and color bar
- Clearly label predicted vs actual classes

```
[47]: import numpy as np
      import seaborn as sns
      import matplotlib.pyplot as plt
      from sklearn.metrics import confusion_matrix

      y_true = np.array([0,1,0,1,2,2,0,1,2,0])
      y_pred = np.array([0,0,0,1,2,1,0,1,2,2])

      cm = confusion_matrix(y_true, y_pred)
      sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=True)
      plt.xlabel("Predicted Class")
      plt.ylabel("Actual Class")
      plt.title("Confusion Matrix")
      plt.show()
```



Lab 12

Question # 16: Create a `Pizza` class with size, crust type, and toppings. Design an ordering system that calculates prices based on selections and allows custom pizza creation with various topping combinations.

```python
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.available = True

class Library:
    def __init__(self):
        self.books = []

    def add_book(self, book):
        self.books.append(book)

    def borrow_book(self, title):
        for b in self.books:
            if b.title == title and b.available:
                b.available = False
                print(f"You borrowed '{b.title}'")
                return
        print("Book not available.")

lib = Library()
lib.add_book(Book("Python Basics", "John"))
lib.borrow_book("Python Basics")
```

```
You borrowed 'Python Basics'
```

Question # 17: Design classes for `Car` (model, year, color, price) and `CarManufacturer` that can produce different car models with various features. Include quality control checks and production tracking.

```
class Room:
    def __init__(self, number, price):
        self.number = number
        self.price = price
        self.available = True

class Hotel:
    def __init__(self):
        self.rooms = []

    def add_room(self, room):
        self.rooms.append(room)

    def book_room(self, number, days):
        for r in self.rooms:
            if r.number == number and r.available:
                total = r.price * days
                r.available = False
                print(f"Room {number} booked for Rs.{total}")
                return
        print("Room not available.")

h = Hotel()
h.add_room(Room(101, 3000))
h.book_room(101, 2)
```

```
Room 101 booked for Rs.6000
```

Question # 18: Create `Teacher` and `Student` classes, and a `Classroom` class that manages assignments, grades, and attendance. Implement functionality to track student performance and generate class reports.

```
class Ride:
    def __init__(self, driver, distance):
        self.driver = driver
        self.distance = distance

    def fare(self):
        return self.distance * 50

class Passenger:
    def __init__(self, name):
        self.name = name

    def book_ride(self, ride):
        print(f"{self.name} booked a ride with {ride.driver}")
        print(f"Fare: Rs.{ride.fare()}")

r1 = Ride("Ali", 10)
p1 = Passenger("Sara")
p1.book_ride(r1)
```

```
Sara booked a ride with Ali
Fare: Rs.500
```

Question # 19: Design `Flight` classes (flight number, destination, departure time, capacity) and a `Passenger` class. Create a booking system that reserves seats and manages passenger manifests.

```
class Patient:
    def __init__(self, name, disease):
        self.name = name
        self.disease = disease

class Doctor:
    def __init__(self, name):
        self.name = name

class Hospital:
    def __init__(self):
        self.records = []

    def admit(self, patient, doctor):
        self.records.append((patient.name, doctor.name))
        print(f"{patient.name} admitted under Dr.{doctor.name}")

p = Patient("Ali", "Fever")
d = Doctor("Ahmed")
h = Hospital()
h.admit(p, d)
```

```
Ali admitted under Dr.Ahmed
```

Question # 20: Create classes for `SmartDevice` (base class) and specific devices like `SmartLight`, `Thermostat`, and `SecurityCamera`. Implement a `SmartHome` class that can control all devices and create automation routines.

```
class Account:
    def __init__(self, name, balance=0):
        self.name = name
        self.balance = balance

    def deposit(self, amt):
        self.balance += amt
        print("Deposit successful.")

    def withdraw(self, amt):
        if amt <= self.balance:
            self.balance -= amt
            print("Withdrawal successful.")
        else:
            print("Insufficient funds.")

a1 = Account("Ali", 1000)
a1.deposit(500)
a1.withdraw(300)
```

```
Deposit successful.
Withdrawal successful.
```